

Django WebSocket + Channels -- Complete Notes

1. WebSocket ?

WebSocket communication protocol client server- **persistent, full-duplex** (bidirectional) connection

connection client server data

:

```text

Client Server

Persistent

Connection

```

WebSocket- **HTTP request**

2. Django- WebSocket ?

Django application- WebSocket **real-time communication**-

, server data page reload client- data

Common Use Cases

```text

Live Chat

Real-time Notifications

Real-time Dashboard

Online User Status

Live Tracking

Gaming

Live Updates

Collaborative Applications

...

### Example

user chat message :

``text

User A

WebSocket

Server

WebSocket

User B

...

User B- browser refresh

---

# 3. WebSocket ?

HTTP connection- WebSocket connection establish handshake

connection WebSocket protocol- upgrade

Basic flow:

```text

Client

WebSocket Handshake

Server

Connection Established

WebSocket Connection

Client

Server

Client

...

Connection message

4. HTTP vs WebSocket

| | HTTP | WebSocket |

| ----- | ----- | ----- |

| Connection | Request/response | Persistent connection |

| Communication | Request Response | Bidirectional |

| Server message | request- response | |

| Real-time communication | polling/other techniques | |

| Common use | REST API, Web pages | Chat, notifications, live updates |

HTTP

```text

Client

Request

Server

Response

Client

...

request:

```text

Client

Request

Server

Response

Client

...

WebSocket

```text

Client

Persistent Connection

Server

...

Connection established    message

---

## # 5. Django WebSocket Support ?

Traditional Django request/response framework HTTP-

WebSocket asynchronous protocols **\*\*Django Channels\*\***

### ## Django Channels

**\*\*Django Channels\*\*** Django application- WebSocket asynchronous communication use case handle

Architecture:

```text

Browser

WebSocket

ASGI

Django Channels

Consumer

```

---

## # 6. ASGI ?

Django- traditional WSGI interface synchronous HTTP application-

**\*\*ASGI (Asynchronous Server Gateway Interface)\*\*** asynchronous applications WebSocket- protocols support

Architecture:

```text

Client

ASGI Server

Django / Channels

Consumer

...

Django project- :

```
```python
```

```
ASGI_APPLICATION = "project_name.asgi.application"
```

```
```
```

```
---
```

7. Django Channels Install

Virtual environment activate :

```
```bash
```

```
pip install channels
```

```
```
```

Redis channel layer :

```
```bash
```

```
pip install channels-redis
```

```
```
```

```
---
```

8. `settings.py` Configuration

`settings.py`-:

```
```python
```

```
INSTALLED_APPS = [
```

```
...
```

```
"channels",
```

```
]
```

```
```
```

```
:
```

```
```python
```

```
ASGI_APPLICATION = "project_name.asgi.application"
```

```
```
```

```
#### Important
```

`project\_name`- Django project package-

project- :

```
```text
```

```
config
```

```
```
```

```
:
```

```
```python
```

```
ASGI_APPLICATION = "config.asgi.application"
```

```
```
```

```
---
```

9. `asgi.py` Configuration

Project-:

```
```text
```

```
project_name/asgi.py
```

```
```
```

```
:
```

```
```python
```

```
import os
```

```
from channels.auth import AuthMiddlewareStack
```

```
from channels.routing import ProtocolTypeRouter, URLRouter
```

```
from django.core.asgi import get_asgi_application
```

```
import app.routing
```

```
os.environ.setdefault(
```

```
"DJANGO_SETTINGS_MODULE",
```

```
"project_name.settings",
```

```
)
```

```
application = ProtocolTypeRouter(
```

```
{
```

```
"http": get_asgi_application(),
```

```
"websocket": AuthMiddlewareStack(
```

```
URLRouter(
```

```
app.routing.websocket_urlpatterns
```

```
)
,
}
)
...
```

```

```

# 10. `ProtocolTypeRouter` ?

`ProtocolTypeRouter` incoming connection- protocol request application-

Example:

```
```text
```

Client

ProtocolTypeRouter

/\

/\

HTTP WebSocket

Django Channels

```
...
```

Configuration:

```
```python
```

ProtocolTypeRouter(  
{

{

"http": get\_asgi\_application(),

```
"websocket": ...,
```

```
}
```

```
)
```

```
...
```

```
:
```

```
```text
```

HTTP Django

WebSocket Channels

```
...
```

```
---
```

11. `AuthMiddlewareStack` ?

```
```python
```

```
AuthMiddlewareStack(...)
```

```
...
```

WebSocket connection- Django authentication/session information

consumer- authenticated user :

```
```python
```

```
self.scope["user"]
```

```
...
```

Example:

```
```python
```

```
def connect(self):
```

```
user = self.scope["user"]
```

```
print(user)
```

```
self.accept()
```

```
...
```

```

```

## # 12. Routing

App- :

```
```text
```

```
app/
```

```
routing.py
```

```
consumers.py
```

```
...
```

```
```
```

```
`routing.py`:
```

```
```python
```

```
from django.urls import path
```

```
from . import consumers
```

```
websocket_urlpatterns = [
```

```
path(
```

```
"ws/chat/",
```

```
consumers.ChatConsumer.as_asgi(),
```

```
),
```

]

...

:

```text

/ws/chat/

...

WebSocket endpoint

---

# 13. Consumer ?

**\*\*Consumer\*\*** WebSocket- application logic connection, incoming message disconnect handle

:

```text

WebSocket Request

Consumer

Connection / Message Logic

...

14. `WebsocketConsumer`

basic synchronous consumer:

```python

```

import json

from channels.generic.websocket import WebsocketConsumer

class ChatConsumer(WebsocketConsumer):

 def connect(self):
 self.accept()

 def disconnect(self, close_code):
 pass

 def receive(self, text_data):
 data = json.loads(text_data)

 message = data["message"]

 self.send(
 text_data=json.dumps(
 {
 "message": message,
 }
)
)
 ...

15. `connect()` Method

```python
def connect(self):

```

```
self.accept()
```

```
'''
```

Client WebSocket connection :

```
'''text
```

Client

```
connect()
```

```
accept()
```

Connection Established

```
'''
```

```
`accept()` connection
```

```
---
```

16. `receive()` Method

Client message :

```
'''python
```

```
def receive(self, text_data):
```

```
'''
```

method call

Example incoming JSON:

```
'''json
```

```
{
```

```
"message": "Hello Server"
```

```
}
```

```

Python-:

```python

data = json.loads(text_data)

message = data["message"]

```

---

# 17. `send()` Method

Server client- message :

```python

self.send(

text_data=json.dumps(

{

"message": message,

}

)

)

```

Flow:

```text

Client

"Hello"

Server

"Hello"

Client

18. `disconnect()` Method

Client connection :

```python

def disconnect(self, close\_code):

pass

---

method call

`close\_code` connection code

---

# 19. Frontend JavaScript

Browser WebSocket connection :

```javascript

const socket = new WebSocket(

"ws://127.0.0.1:8000/ws/chat/"

);

20. `onopen`

Connection successfully established :

```
````javascript
socket.onopen = function () {
 socket.send(
 JSON.stringify({
 message: "Hello Server",
 })
);
};
...

```

Flow:

```
````text
```

Browser

WebSocket Connect

Server accepts

onopen()

Send message

...

21. `onmessage`

Server message :

```
```javascript
socket.onmessage = function (event) {
 const data = JSON.parse(event.data);

 console.log(data.message);
};
```
```

Server :

```
```json
{
 "message": "Hello Client"
}
```
```

Browser:

```
```text
Hello Client
```
```

22. Complete Basic WebSocket Flow

```
```text
Browser

WebSocket
```

Django ASGI

Protocol Router

Consumer

Receive Send

Message Response

Browser

...

---

# 23. Complete Example

## `consumers.py`

```python

import json

from channels.generic.websocket import WebsocketConsumer

class ChatConsumer(WebsocketConsumer):

def connect(self):

self.accept()

def disconnect(self, close_code):

pass

```

def receive(self, text_data):
    data = json.loads(text_data)

    message = data["message"]

    self.send(
        text_data=json.dumps(
            {
                "message": message,
            }
        )
    )
    ...

## `routing.py`

```python
from django.urls import path

from . import consumers

websocket_urlpatterns = [
 path(
 "ws/chat/",
 consumers.ChatConsumer.as_asgi(),
),
]
...

`asgi.py`

```

```

```python
import os

from channels.auth import AuthMiddlewareStack
from channels.routing import ProtocolTypeRouter, URLRouter
from django.core.asgi import get_asgi_application

import app.routing

os.environ.setdefault(
    "DJANGO_SETTINGS_MODULE",
    "project_name.settings",
)

application = ProtocolTypeRouter(
    {
        "http": get_asgi_application(),

        "websocket": AuthMiddlewareStack(
            URLRouter(
                app.routing.websocket_urlpatterns
            )
        ),
    }
)
```

`settings.py`

```python

```

```

INSTALLED_APPS = [

# ...

"channels",

]

ASGI_APPLICATION = "project_name.asgi.application"

...

## Frontend JavaScript

```javascript
const socket = new WebSocket(
"ws://127.0.0.1:8000/ws/chat/"
);

socket.onopen = function () {
socket.send(
JSON.stringify({
message: "Hello Server",
})
);
};

socket.onmessage = function (event) {
const data = JSON.parse(event.data);

console.log(data.message);
};
...

```

## # 24. Important Limitation of This Basic Example

`ChatConsumer` \*\* client message , client- response \*\*

:

```text

User A

Consumer

User A

```

real multi-user chat

Multi-user chat- \*\*Channel Layer Groups\*\*

---

## # 25. Channel Layer ?

Channel Layer communication system consumer/process- message

Production environment- Redis- Channel Layer common

Architecture:

```text

Consumer A

Channel Layer

Redis

Channel Layer

Consumer B

...

26. Redis + Channels

Install:

```
```bash
```

```
pip install channels-redis
```

```
```
```

`settings.py`:

```
```python
```

```
CHANNEL_LAYERS = {
```

```
"default": {
```

```
"BACKEND": (
```

```
"channels_redis.core.RedisChannelLayer"
```

```
),
```

```
"CONFIG": {
```

```
"hosts": [
```

```
("127.0.0.1", 6379),
```

```
],
```

```
},
```

```
},
```

```
}
```

...

Architecture:

```text

Browser

WebSocket

Django Channels

Redis Channel Layer

Other Consumers

...

27. Groups

chat room- users- group-

Example:

```text

Chat Room: "room1"

Redis

User A User B User C

...

User A message :

```text

User A

Consumer

room1 group

User B

User C

...

User A- group- message , application logic broadcast

28. Group-based Chat Consumer

simplified synchronous example:

```
```python
```

```
import json
```

```
from asgiref.sync import async_to_sync
```

```
from channels.generic.websocket import WebsocketConsumer
```

```
class ChatConsumer(WebsocketConsumer):
```

```
 def connect(self):
```

```
 self.room_group_name = "chat_room"
```

```
 async_to_sync(
```

```
 self.channel_layer.group_add
```

```
)(
```

```
 self.room_group_name,
```

```
self.channel_name,
)

self.accept()

def disconnect(self, close_code):

 async_to_sync(
 self.channel_layer.group_discard
)(
 self.room_group_name,
 self.channel_name,
)

def receive(self, text_data):

 data = json.loads(text_data)

 message = data["message"]

 async_to_sync(
 self.channel_layer.group_send
)(
 self.room_group_name,
 {
 "type": "chat_message",
 "message": message,
 },
)

def chat_message(self, event):
```

```
message = event["message"]
```

```
self.send(
```

```
text_data=json.dumps(
```

```
{
```

```
"message": message,
```

```
}
```

```
)
```

```
)
```

```
'''
```

```
:
```

```
```python
```

```
group_add()
```

```
'''
```

```
consumer- group-
```

```
```python
```

```
group_discard()
```

```
'''
```

```
consumer- group
```

```
```python
```

```
group_send()
```

```
'''
```

```
group- consumers- message
```

```
---
```

29. Async Consumer

Channels- asynchronous consumer-

Example:

```
```python
import json

from channels.generic.websocket import AsyncWebsocketConsumer

class ChatConsumer(AsyncWebsocketConsumer):

 async def connect(self):
 await self.accept()

 async def disconnect(self, close_code):
 pass

 async def receive(self, text_data):
 data = json.loads(text_data)

 message = data["message"]

 await self.send(
 text_data=json.dumps(
 {
 "message": message,
 }
)
)
```
```

30. Sync vs Async Consumer

`WebSocketConsumer`

```python

class ChatConsumer(WebSocketConsumer):

...

Synchronous style

### `AsyncWebSocketConsumer`

```python

class ChatConsumer(AsyncWebSocketConsumer):

...

Asynchronous style

Async consumer useful application- asynchronous operations

> `AsyncWebSocketConsumer` performance automatically -- Async code- workload
I/O operations asynchronous handle

31. WebSocket URL

Development-:

```javascript

const socket = new WebSocket(

"ws://127.0.0.1:8000/ws/chat/"

);

...

Production- HTTPS :

```
```javascript
```

```
const socket = new WebSocket(
```

```
"wss://example.com/ws/chat/"
```

```
);
```

```
...
```

Difference

```
```text
```

ws://

WebSocket

wss://

Secure WebSocket

```
...
```

`wss://` TLS/HTTPS- encrypted connection

---

# 32. Real-Time Notification Example

notification system:

```
```text
```

Database Event

Django

Channel Layer

Redis

WebSocket

Browser

New Notification

...

Browser page reload notification

33. Online User Status

WebSocket connection online/offline status track

Example:

```text

User A connects

WebSocket connected

User A = Online

...

Connection :

```text

WebSocket disconnected

User A = Offline

...

Production system- online presence design reconnects, multiple tabs/devices, timeouts stale connections

34. Production Architecture

Django/Channels instance Redis Channel Layer

``text

Load Balancer

Django/Channels 1 Django/Channels 2

Redis

Channel Layer

Client A Client B

Redis helps different application instances communicate through the channel layer.

35. Important Production Considerations

Production WebSocket systems should consider:

``text

Redis

ASGI server

Reverse proxy

TLS / WSS

Authentication

Authorization

Connection management

Reconnection handling

Scaling

Monitoring

Logging

Resource limits

...

WebSocket application- HTTP request- ****long-lived connections**** manage

36. WebSocket Security

WebSocket application- authentication authorization

connection establish user- resource access

Example:

```text

Authenticated User

WebSocket Connection

Authorization Check

Allowed Resource

...

Sensitive chat room private notification system user room/resource access server-side verify

---

## # 37. Common Mistakes

### ## Mistake 1 -- `channels` install

:

```text

channels

ASGI configuration

routing

consumer

frontend WebSocket code

```

multi-instance/group communication- :

```text

channels-redis

Redis

```

---

### ## Mistake 2 -- Redis multi-instance architecture

Single-process development- simple setup production multi-instance Channels deployment- shared channel layer

Redis common choice

---

## Mistake 3 -- WebSocket HTTP

HTTP:

```text

Request Response

```

WebSocket:

```text

Persistent Connection

Bidirectional Messages

```

---

## Mistake 4 -- Basic consumer- full chat system

code:

```python

self.send(...)

```

current WebSocket connection- response

Multi-user chat- :

```text

Channel Layer

+

Groups

+

Redis

...

38. Complete Architecture

```text

Browser

WebSocket

Reverse Proxy / Server

Django ASGI / Channels

Consumer Consumer

Redis Channel Layer

Client A Client B

...

---

## # 39. Quick Revision

---

### ### WebSocket

```text

Persistent

Bidirectional

Real-time communication

```

### ### Django Channels

```text

Django

ASGI

Channels

WebSocket

```

### ### Consumer

```text

WebSocket connection

Consumer

Application logic

```

### ### Channel Layer

```text

Consumer

Channel Layer

Other Consumers

...

Redis

``text

Channels

Redis Channel Layer

Shared communication

...

Group

``text

Group

User A

User B

User C

...

A message can be broadcast to all members of the group.

40. One-Line Definitions

``text

WebSocket

A persistent, bidirectional communication protocol.

Django Channels

A Django project that adds support for WebSockets and other non-HTTP/asynchronous communication patterns.

ASGI

The interface used by Python applications/frameworks to communicate with asynchronous-capable servers and protocols.

Consumer

The application component that handles a WebSocket connection and its events.

Channel Layer

A communication layer that allows Channels consumers to exchange messages.

Redis

A common backend for the Channels channel layer.

Group

A named collection of channel connections that can receive broadcast messages.

`ws://`

WebSocket connection.

`wss://`

Secure WebSocket connection.

...

Final Mental Model

```text

---

USER BROWSER

WebSocket

Django ASGI

Django Channels

Consumer

Channel Layer

Redis

Consumer A Consumer B

User A User B

...

### Remember:

```text

HTTP

Request / Response

WebSocket

Persistent + Bidirectional

Channels

Django WebSocket handling

Consumer

WebSocket logic

Redis

Shared Channel Layer backend

Group

Broadcast communication

ASGI

WebSocket-capable application interface

...